Lesson8
完善类的设计

主讲老师： 申雪萍

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容（1）

- 面向对象的几个基本原则
  - UML类图简介
  - 面向抽象（接口、抽象类）原则
  - 优先使用组合少用继承原则
  - 开-闭原则
  - 高内聚-低耦合原则
  - 其它原则

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容（2）

- 设计模式简介
- 几个重要的设计模式
  - 策略模式
  - 访问者模式
  - 装饰模式
  - 适配器模式
  - 责任链模式
  - 门面模式
  - 简单工厂模式
  - 工厂方法模式
  - 抽象工厂模式

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

后面讲

- 最终方法、最终类
- Object类
- 内部类
- 匿名类

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

面向对象的几个基本原则

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

必要性（什么才是优秀的软件架构？）

- 了解面向对象设计的基本原则，有助于大家合理使用面向对象语言，编写出：
  - 低耦合、
  - 易维护、
  - 易扩展
  - 和易复用的程序代码。

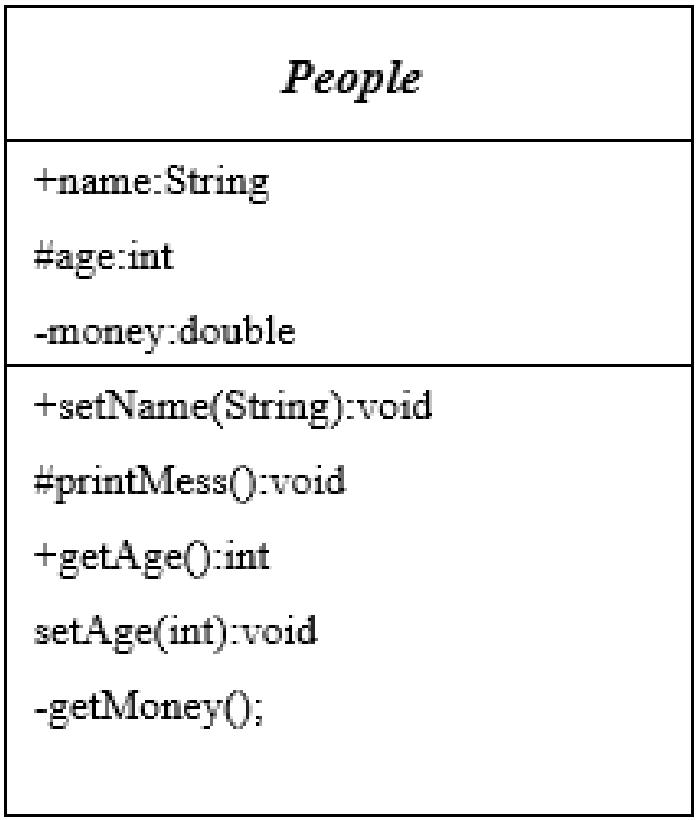
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

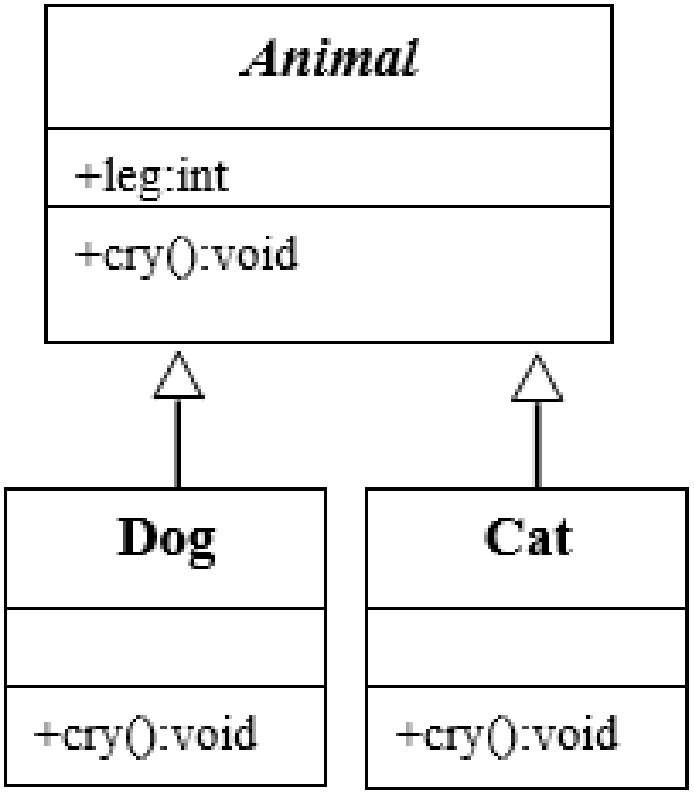
类的UML图

- UML-Unified Modeling Language 统一建模语言，又称标准建模语言。是用来对软件密集系统进行可视化建模的一种语言。UML的定义包括UML语义和UML表示法两个元素。
  - 接口（Interface）
  - 泛化关系（Generalization）
  - 关联关系（Association）
  - 依赖关系（Dependency）
  - 实现关系（Realization）

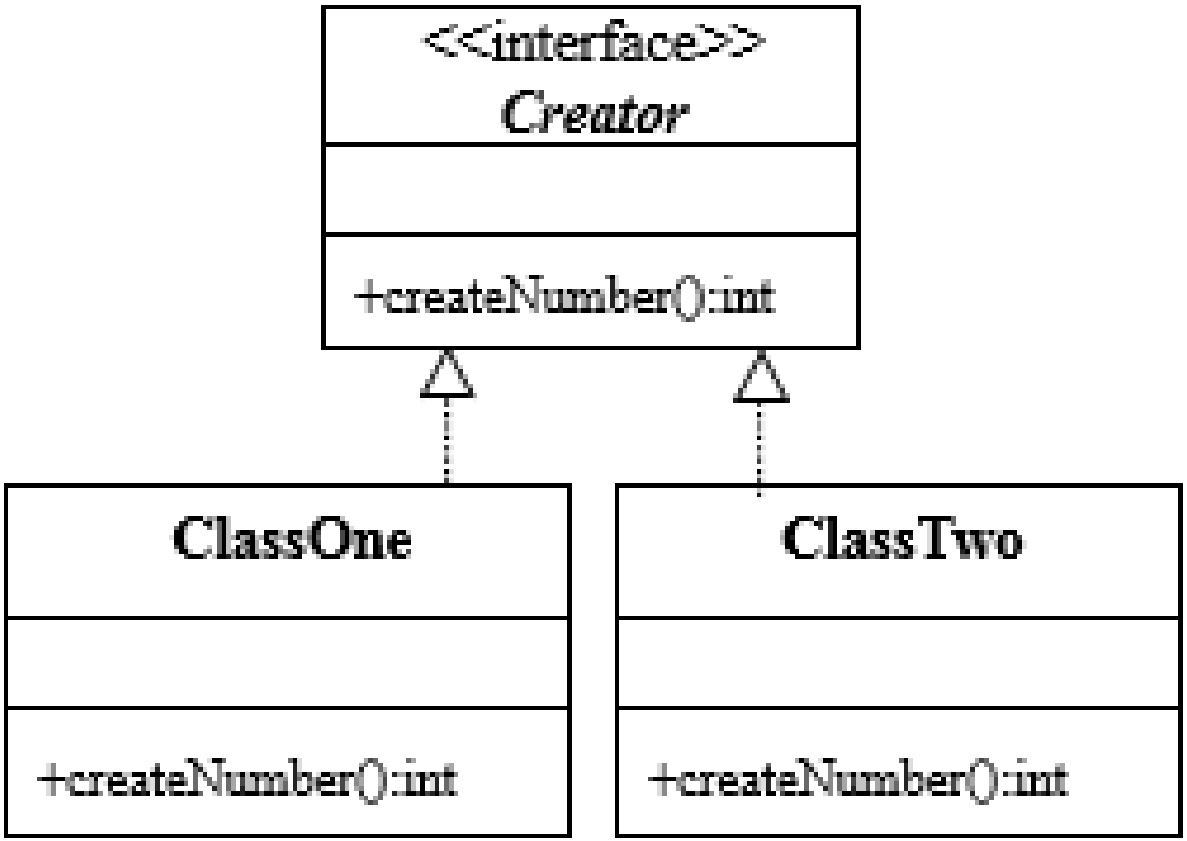
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

类以及类的继承关系

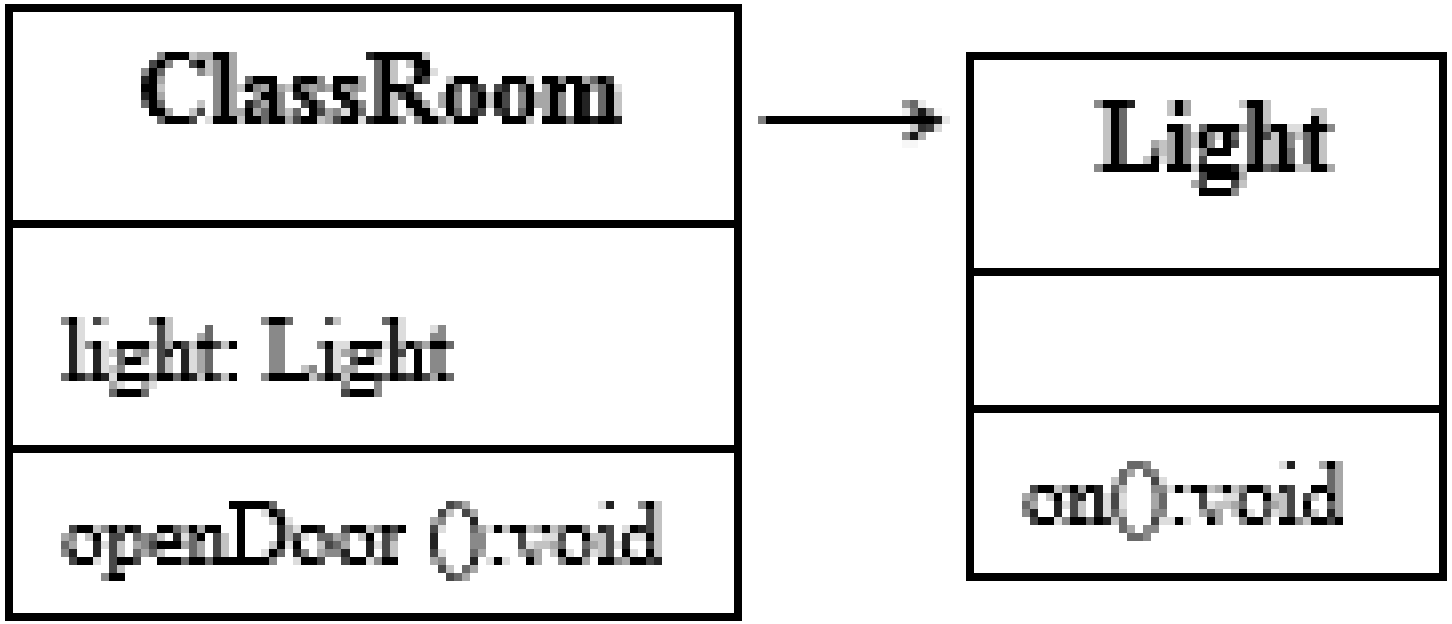

People
+name:String
#age:int
-money:double
+setName(String):void
#printMess():void
+getAge():int
setAge(int):void
-getMoney();


Animal
+leg:int
+cry():void
Dog
+cry():void
Cat
+cry():void

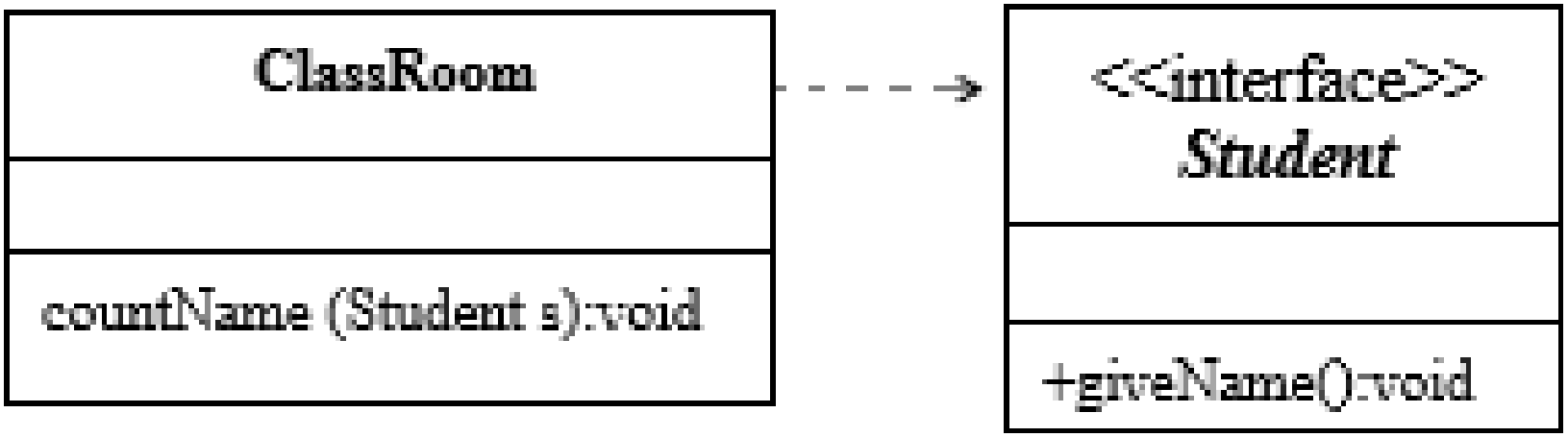
接口和实现关系


<<interface>>
Creator
+createNumber():int
ClassOne
+createNumber():int
ClassTwo
+createNumber():int

关联关系

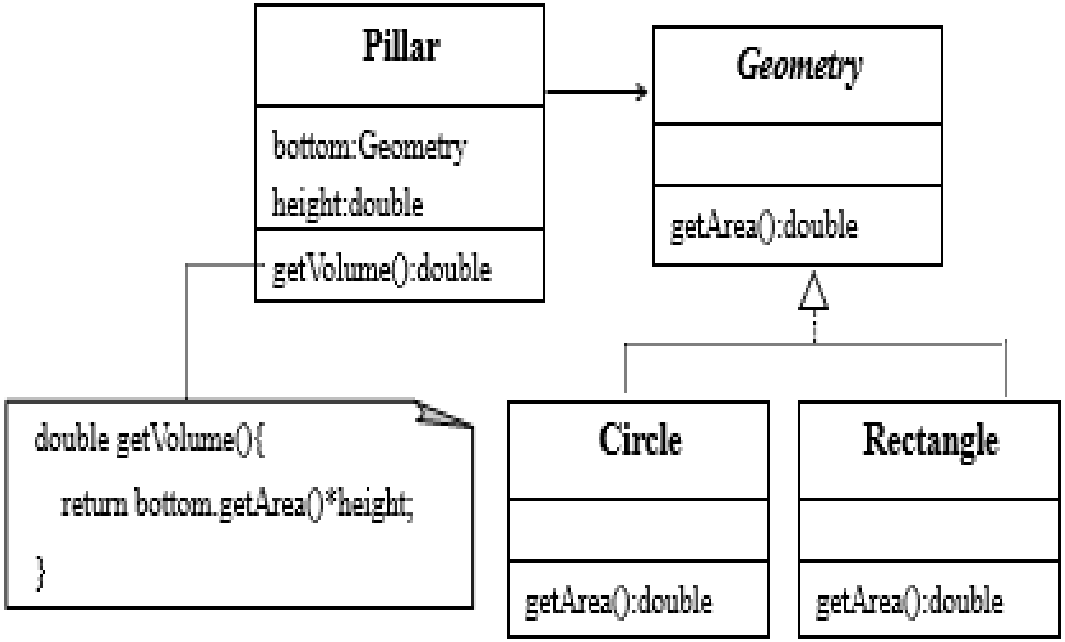


依赖关系


ClassRoom
countName (Student s):void
<<interface>>
Student
+giveName():void

面向接口的编程（面向抽象的原则）（1）

- 面向接口的编程：是面向对象设计（OOD）的非常重要的基本原则之一。
- 它的含义是：使用接口和同类型的组件通讯，即，对于所有完成相同功能的组件，应该抽象出一个接口，它们都实现该接口。



北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

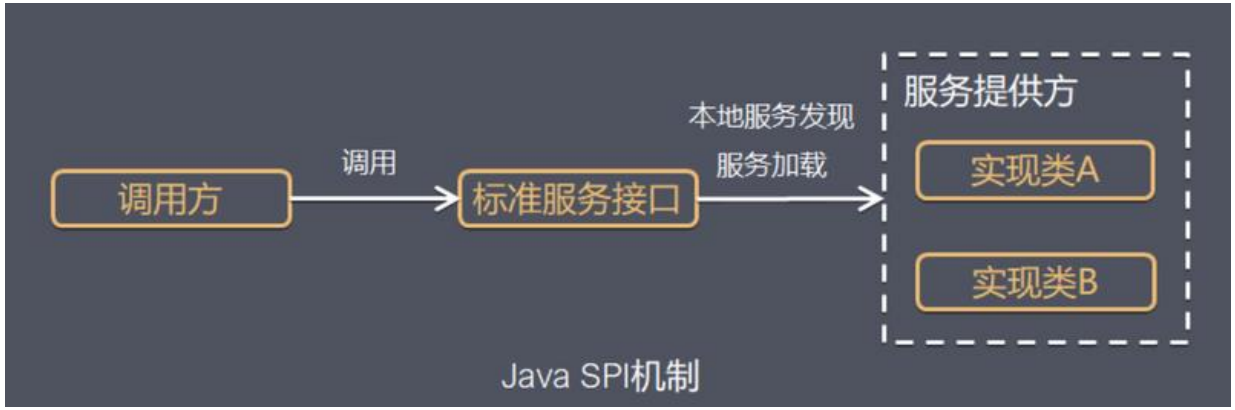
面向接口的编程（2）

- 具体到JAVA中，可以是接口（interface），或者是抽象类（abstract class），所有完成相同功能的组件都实现该接口，或者从该抽象类继承。
- 客户代码只应该和该接口通讯。
  - 降低代码耦合性，提升代码的可维护性
  - 修改一方，不会影响另一方
  - 可复用性提升

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

面向接口的编程（3）

- **场景1：**当我们需要用其它组件完成任务时，只需要替换该接口的实现，代码的其它部分不需要改变。
- **场景2：**当现有的组件不能满足要求时，我们可以创建新的组件，实现该接口，或者，直接对现有的组件进行扩展，由子类去完成扩展的功能。



- **减少了代码耦合，实现了代码复用**

面向抽象原则（抽象类和接口）

- 当设计一个类时，不让该类面向具体的类，而是面向抽象类或接口，即所设计类中的重要数据是抽象类或接口声明的变量，而不是具体类声明的变量
- **案例：**
  - 假设已有Circle类，该类创建的对象circle调用getArea()方法可以计算圆的面积。
  - 现在要设计一个Pillar类（柱类），该类的对象调用getVolume()方法可以计算柱体的体积

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

面向抽象原则（抽象类和接口）

```
public class Pillar {
    Circle  bottom;    //将Circle对象作为成员，bottom是用具体类声明的变量
    double height;
    Pillar (Circle bottom,double height) {
        this.bottom=bottom;this.height=height;
    }
    public double getVolume() {
        return bottom.getArea()*height;
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

面向抽象编程的思想

- 如果不涉及用户需求的变化，上面Pillar类的设计没有什么不妥，但是在某个时候，用户希望Pillar类能创建出底是三角形、正方形、椭圆等的柱体。显然上述Pillar类无法创建出这样的柱体，即上述设计的Pillar类不能应对用户的这种需求。

- 因此，我们在设计Pillar类时**不应当让它的底是某个具体类声明的变量**，一旦这样做，Pillar类就依赖该具体类，缺乏弹性，难以应对需求的变化。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

面向抽象编程的思想

- 首先编写一个抽象类Geometry（或接口），该抽象类（接口）中定义了一个抽象的getArea()方法

```
public abstract class Geometry { //如果使用接口需用interface来定义Geometry。
    public abstract double getArea();
}
```

```
public class Pillar {
    Geometry  bottom;  //bottom是抽象类Geometry声明的变量
    double height;
    Pillar (Geometry bottom,double height) {
        this.bottom = bottom; this.height=height;
    }
    public double getVolume() {
        return bottom.getArea()*height;
    }
}
```

Pillar类的设计不再依赖具体类，而是面向Geometry类

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

面向抽象编程的思想

```
public class Circle extends Geometry
{
    double r;
    Circle(double r) {
        this.r=r;
    }
    public double getArea() {
        return(3.14*r*r);
    }
}
```

```
public class Rectangle extends Geometry {
    double a,b;
    Lader(double a,double b) {
        this.a=a; this.b=b;
    }
    public double getArea() {
        return a*b;
    }
}
```

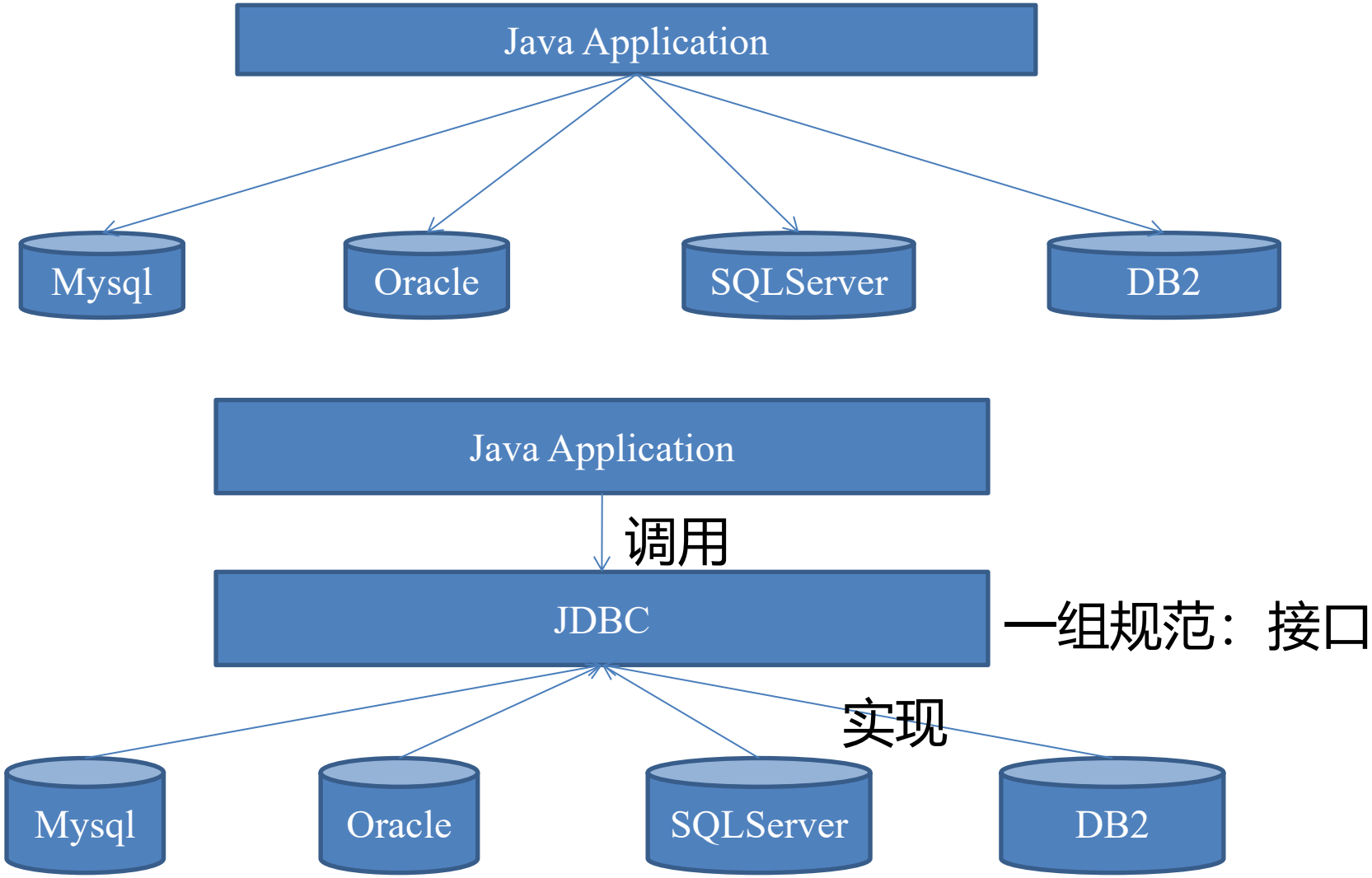

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```
public class Application{
    public static void main(String args[]){
        Pillar pillar;
        Geometry bottom;
        bottom=new Rectangle(12,22,100);
        pillar = new Pillar (bottom,58);  //pillar是具有矩形底的柱体
        System.out.println("矩形底的柱体的体积"+pillar.getVolume());
        bottom=new Circle(10);
        pillar = new Pillar (bottom,58); //pillar是具有圆形底的柱体
        System.out.println("圆形底的柱体的体积"+pillar.getVolume());
    }
}
```

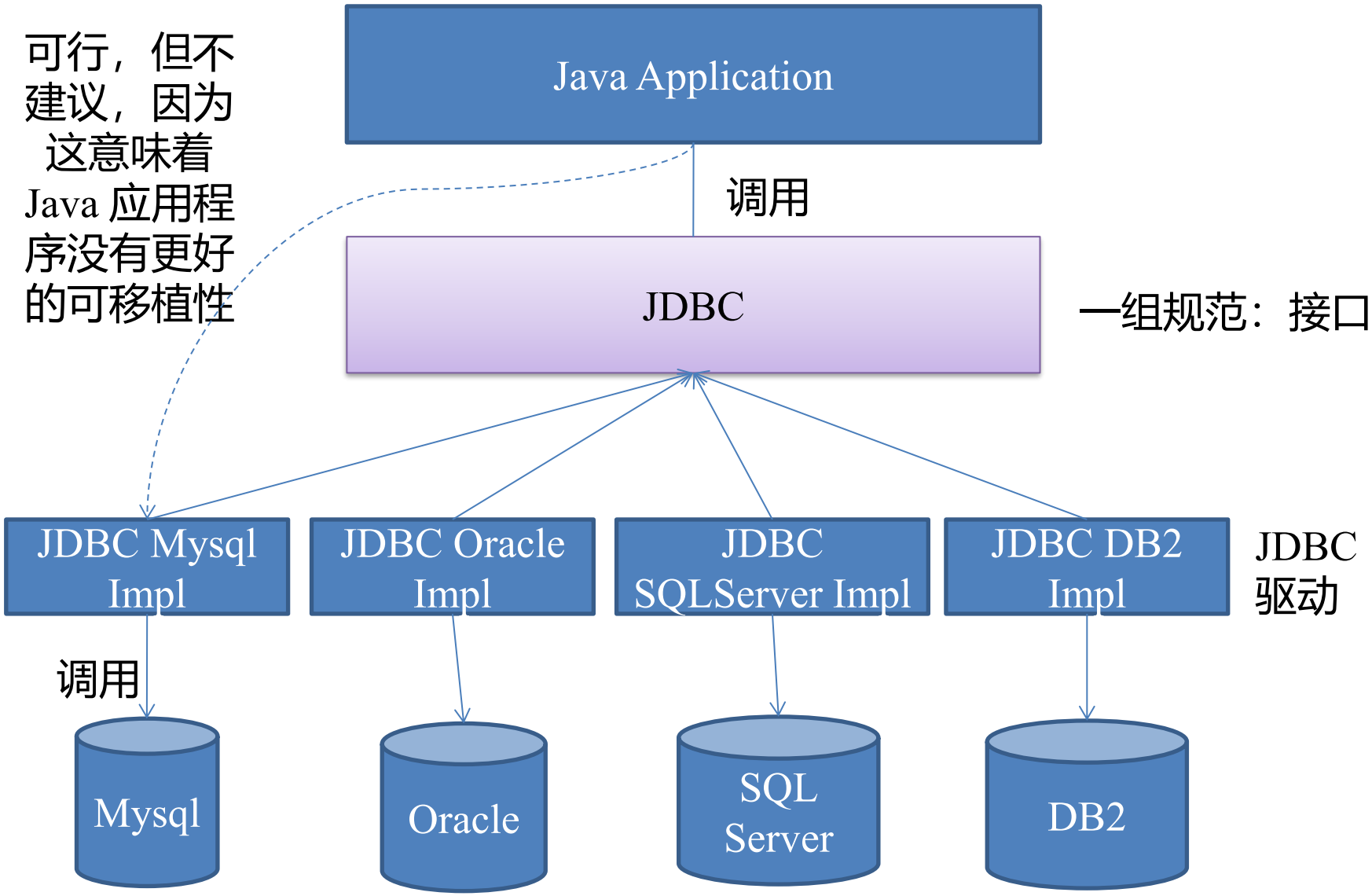
Pillar类不再依赖具体类，因此每当系统增加新的Geometry的子类时，比如增加一个Triangle子类，那么不需要修改Pillar类的任何代码，就可以使用Pillar创建出具有三角形底的柱体。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

JDBC(Java Dadabase Connectivity)

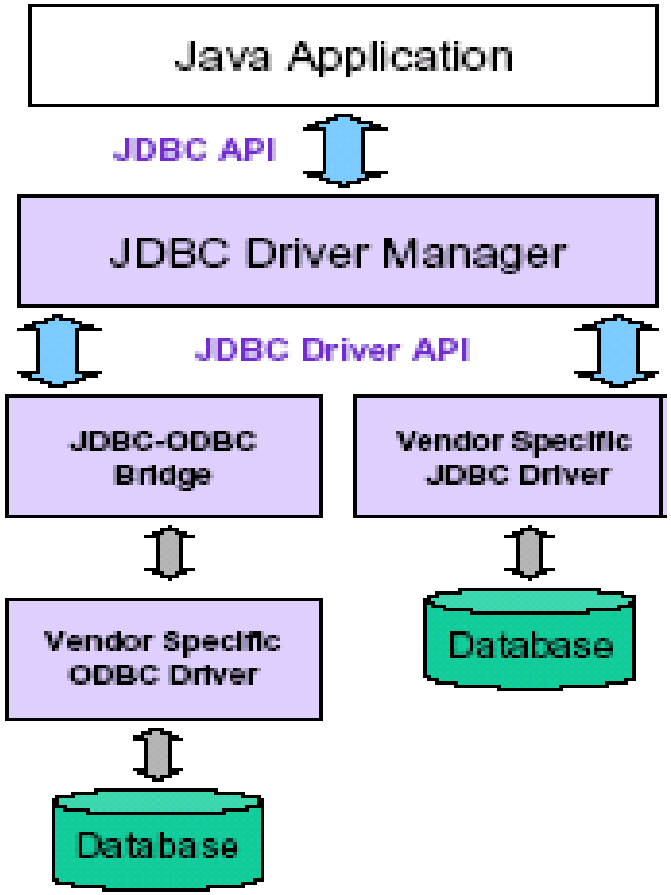


JDBC(Java Dadabase Connectivity)

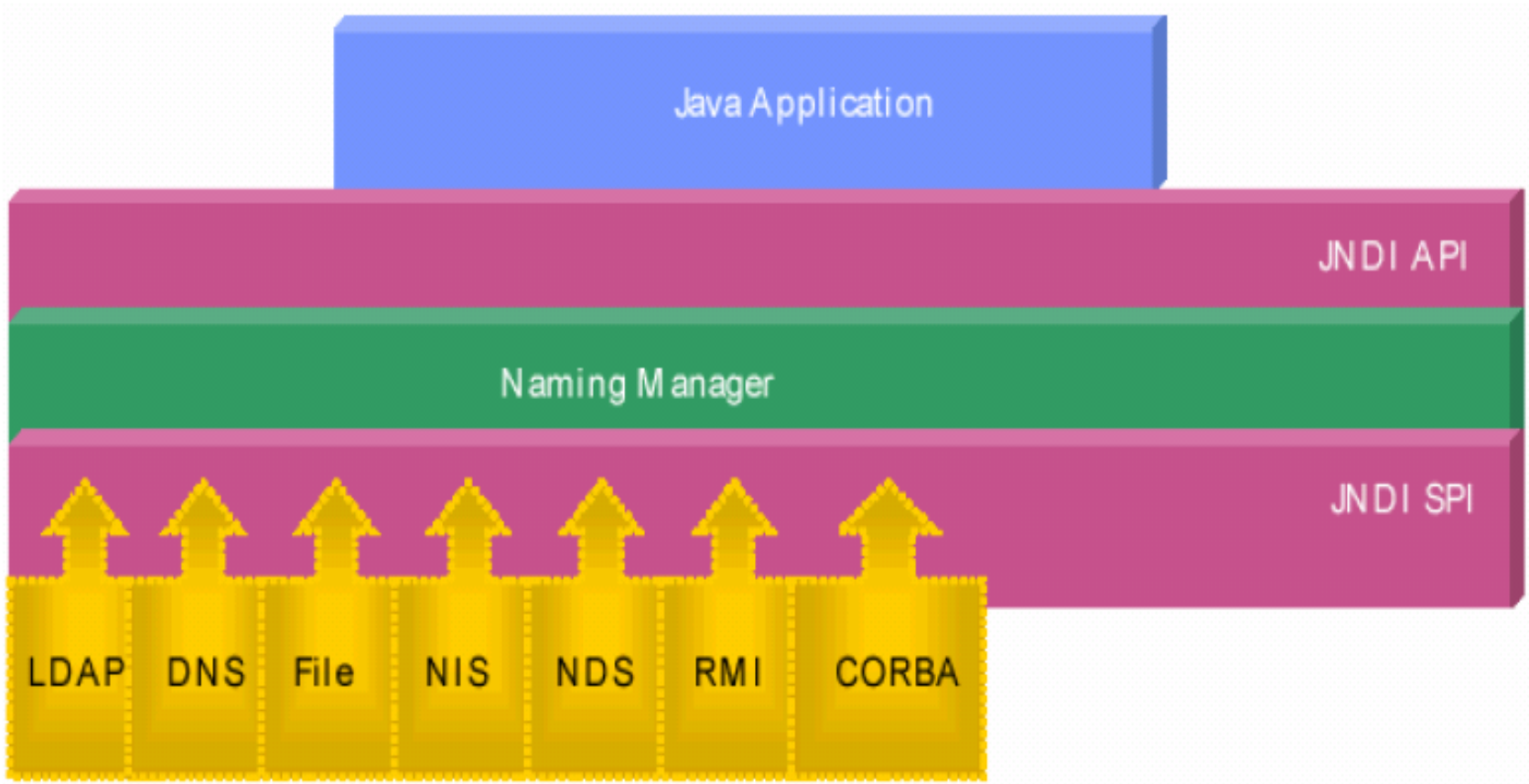


JDBC的组成

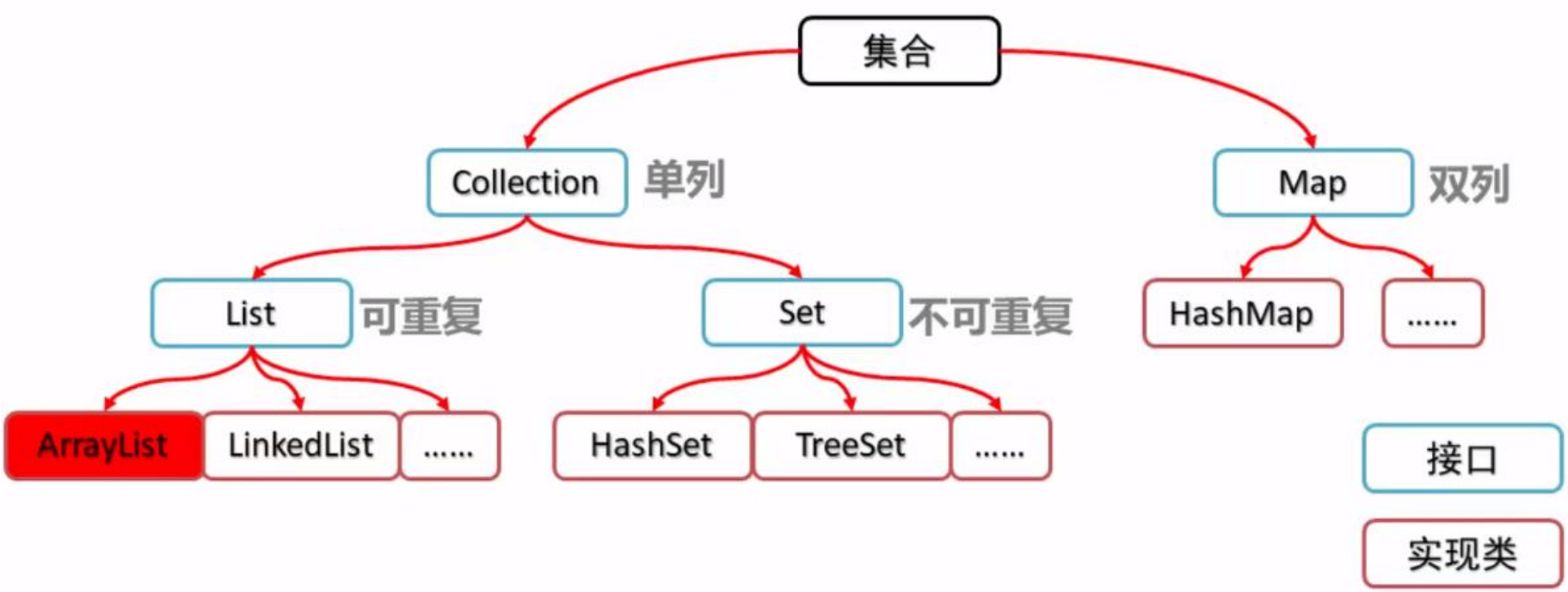
- JDBC有两部分组成：
  - JDBC API: a purely Java-based API
  - JDBC Driver API
- JDBC Driver Manager:which communicates with vendor-specific drivers that perform the real communication with the database.



JNDI(Java Naming and Directory Interface)体系结构



数学中的集合


集合
Collection
单列
Map
双列
List
可重复
Set
不可重复
HashMap
.....
ArrayList
LinkedList
.....
HashSet
TreeSet
.....
接口
实现类

优先使用组合少用继承原则

- 方法复用的两种最常用的技术就是类继承和对象组合
- 通过继承复用方法的缺点是：
  - 继承破坏了封装性，通过继承进行复用也称**“白盒”复用**，其缺点是父类的内部细节对于子类而言是可见的。
  - 子类和父类的关系是**强耦合关系**，也就是说当父类的方法的行为更改时，必然导致子类发生变化
  - 子类从父类继承的方法在编译时刻就确定下来了，所以**无法在运行期间改变从父类继承的方法的行为**。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

组合与复用

- 组合对象来复用方法的优点是：
  - 对象组合是类继承之外的另一种复用选择。新的更复杂的功能可以通过组合对象来获得。
  - 对象组合要求对象具有良好定义的接口。这种复用风格被称为**黑箱复用(black-box reuse)**，因为被组合的对象的内部细节是不可见的,对象只以"黑箱"的形式出现。
- **对象与所包含的对象属于弱耦合关系**，因为，如果修改当前对象所包含的对象的类的代码，不必修改当前对象的类的代码。
- 当前对象可以在**运行时刻动态指定所包含的对象**。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```
public abstract class Person {
   public abstract String getMess();
}
```

```
public  class Driver1 extends Person {
   public  String getMess(){
        return "我是中国驾驶员";
   }
}
```

```
public  class Driver5 extends Person {
   public  String getMess(){
        return "我是女驾驶员";
   }
}
```

```
public  class Driver7 extends Person {
   public  String getMess(){
        return "我是男驾驶员";
   }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
public class Car {
    Person person;    //组合驾驶员
    public void setPerson(Person p) {
        person = p;
    }
    public void show() {
        if(person == null) {
            System.out.println("目前没人驾驶汽车.");
        }
        else {
            System.out.println("目前驾驶员是:"+person.getMess());
        }
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

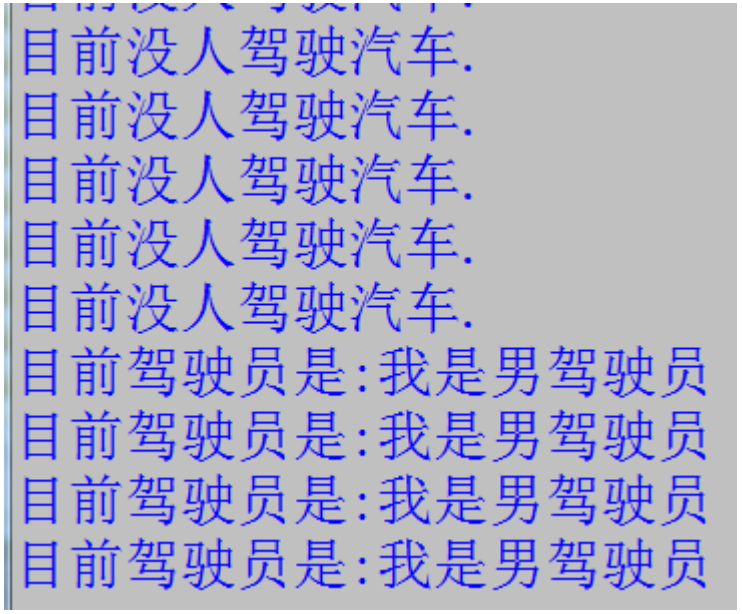
```
public class MainClass {
  public static void main(String arg[]) {
    Car car =new Car();
    int i=1;
    while(true) {
      try{
        car.show();
        Thread.sleep(2000);  //每隔2000毫秒更换驾驶员
        Class<?> cs=Class.forName("Driver"+i);
        Person p=(Person)cs.getDeclaredConstructor().newInstance();
        //如果没有第i个驾驶员就触发异常，跳到catch,即无人驾驶或当前驾驶员继续驾驶:
        car.setPerson(p);      //更换驾驶员
        i++;
      }
      catch(Exception exp){
        i++;
      }
      if(i>10) i=1;          //最多10个驾驶员轮换开车
    }
  }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

模拟汽车动态更换驾驶员（维护代码时，不停止软件的运行）

运行主类MainClass之后，不要关闭程序，然后编写Person的子类。名字可以是Driver1,Driver2…。

重新打开一个命令行编译这些子类，比如Driver7.java。然后就会看到程序运行结果的变化（更换驾驶员）



北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

多用组合少用继承原则

- 之所以提倡多用组合，少用继承，是因为在许多设计中，人们希望系统的类之间尽量是低耦合的关系，而不希望是强偶合关系。即在许多情况下需要避开继承的缺点，而需要组合的优点。
- 怎样合理地使用组合，而不是使用继承来获得方法的复用需要经过一定时间的认真思考、学习和编程实践才能悟出其中的道理。

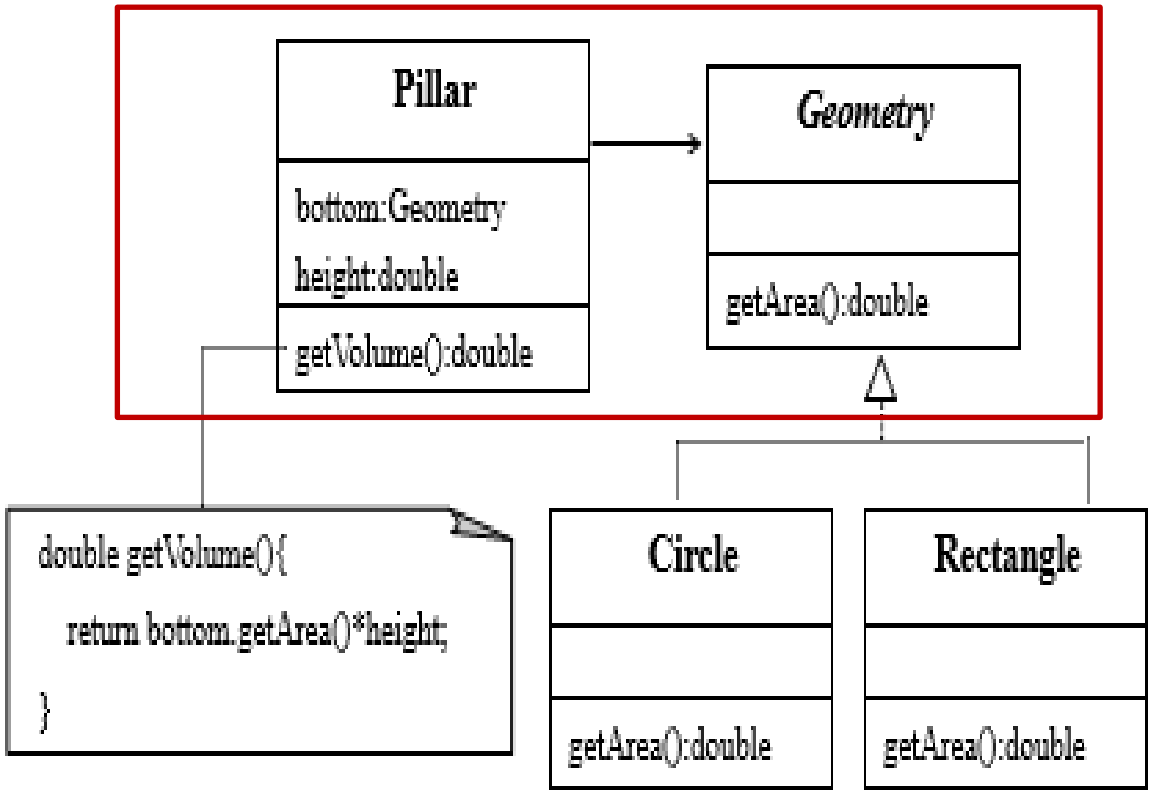
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

开-闭原则

- 所谓 开-闭原则（Open-Closed Principle）就是让你的设计应当对扩展开放，对修改关闭。
- 怎么理解对扩展开放，对修改关闭呢？实际上这句话的本质是指当一个设计中增加新的模块时，不需要修改现有的模块。
- 在给出一个设计时，应当首先考虑到**用户需求的变化**，将应对用户变化的部分设计为对扩展开放，而设计的核心部分是经过精心考虑之后确定下来的基本结构，这部分应当是对修改关闭的，即不能因为用户的需求变化而再发生变化，因为这部分不是用来应对需求变化的

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

开-闭原则



该设计中的Geometry和Pillar类就是系统中对修改关闭的部分，而Geometry的子类是对扩展开放的部分。

经常需要面向抽象来考虑系统的总体设计，不要考虑具体类，这样就容易设计出满足“开-闭原则”的系统。

高内聚-低耦合原则

- 耦合主要描述模块之间的关系
- 内聚主要描述模块内部

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

高内聚

- 一个模块应当尽可能独立完成某个功能。模块内部的元素, 关联性越强, 则内聚越高, 模块单一性则更强。
- 如何判定？
  - 系统存在AB两个模块儿进行交互，如果修改了A模块儿不影响B模块的工作，那么认为A模块儿有足够的内聚。
- **危害：**低内聚的模块代码, 不管是维护, 扩展还是重构都相当麻烦, 难以下手，造成开发效率低下。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

低耦合

- 低耦合：是指软件系统中，模块与模块之间的直接依赖程度低。
  - 模块的独立性高，将使应用程序便于扩展, 维护, 写单元测试。
  - 相反，模块之间存在依赖越多,模块独立性就越差，耦合度就越强,导致一个模块的改动可能会影响到其他模块，因此会极大降低开发效率

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

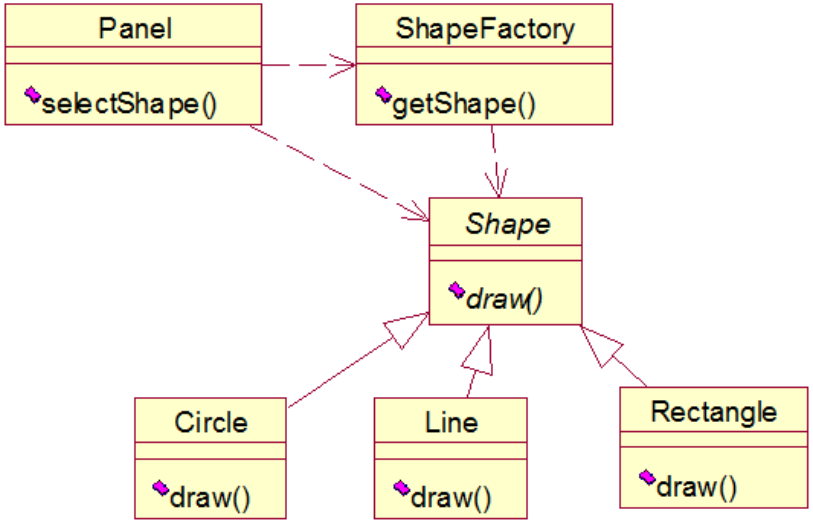
软件设计七大原则

- 单一职责原则: 一个类只负责一个功能领域中的相应职责。
- 开闭原则: 一个软件实体应当对扩展开放，对修改关闭。
- 里氏代换原则: 所有引用基类（父类）的地方必须能透明地使用其子类的对象。
- 合成复用原则：尽量使用组合或者聚合关系实现代码复用，少使用继承。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

软件设计七大原则

- 依赖倒转原则: 抽象不应该依赖于细节, 细节应当依赖于抽象。 换言之, 要针对接口编程, 而不是针对实现编程。
- 接口隔离原则: 使用多个专门的接口, 而不使用单一的总接口, 即客户端不应该依赖那些它不需要的接口。
- 迪米特法则: 一个软件实体应当尽可能少地与其他实体发生相互作用。
  - 例如外观模式, 对外暴露统一接口。



北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

小结

设计原则	一句话归纳	目的
开闭原则	对扩展开放，对修改关闭	降低维护带来的新风险
依赖倒置原则	高层不应该依赖低层，要面向接口编程	更利于代码结构的升级扩展
单一职责原则	一个类只干一件事，实现类要单一	便于理解，提高代码的可读性
接口隔离原则	一个接口只干一件事，接口要精简单一	功能解耦，高聚合、低耦合
迪米特法则	不该知道的不要知道，一个类应该保持对其它对象最少的了解，降低耦合度	只和朋友交流，不和陌生人说话，减少代码臃肿
里氏替换原则	不要破坏继承体系，子类重写方法功能发生改变，不应该影响父类方法的含义	防止继承泛滥
合成复用原则	尽量使用组合或者聚合关系实现代码复用，少使用继承	降低代码耦合

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

小结：

- 在程序设计时，我们应该将程序功能最小化，每个类只干一件事。
- 若在类似功能基础之上添加新功能，则要合理使用继承。
- 对于多方法的调用，要会运用接口，同时合理设置接口功能与数量。
- 最后类与类之间做到低耦合高内聚。

记忆口诀：访问加限制，函数要节俭，依赖不允许，动态加接口，父类要抽象，扩展不更改。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

几个重要的设计模式

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容（2）

- 设计模式简介
- 几个重要的设计模式
  - 门面模式
  - 策略模式
  - 访问者模式
  - 装饰模式
  - 适配器模式
  - 责任链模式
  - 工厂方法模式

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

设计模式简介

- **需求驱动**：设计模式不是为每个人准备的，而是基于业务来选择设计模式，需要时就能想到它。要明白一点，技术永远为业务服务，技术只是满足业务需要的一个工具。
- 我们需要掌握每种设计模式的应用场景、特征、优缺点，以及每种设计模式的关联关系，这样就能够很好地满足日常业务的需要。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

什么是设计模式？

- 一个设计模式（pattern）是针对某一类问题的最佳解决方案，而且已经被成功应用于许多系统的设计中，它解决了在某种特定情景中重复发生的某个问题，即一个设计模式是从许多优秀的软件系统中总结出的成功的可复用的设计方案。
- **学习设计模式的必要性**
  - 列举几个设计模式的目的不仅是要掌握、使用这些模式，更重要的是可以是通过讲解这些设计模式体现面向对象的设计思想，这非常有利于更好地使用面向对象语言解决设计中的诸多问题。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

注意事项

- 在进行设计时，尽可能用最简单的方式满足系统的要求，而不是费尽心机地琢磨如何在这个问题中使用设计模式。
- 事实上，真实世界中的许多设计实例都没有使用过那些所谓的经典设计模式。
- 一个设计可能并不需要使用设计模式就可以很好地满足系统的要求，如果牵强地使用某个设计模式可能会在系统中增加许多额外的类和对象，影响系统的性能。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

设计模式分类

- 创建型：
  - 涉及对象的实例化，这类模式的特点是，不让用户代码依赖于对象的创建或排列方式，避免用户直接使用new运算符创建对象。例如：工厂方法模式
- 行为型
  - 涉及怎样合理地设计对象之间的交互通信，以及怎样合理为对象分配职责，让设计富有弹性，易维护，易复用。例如：策略模式，中介者模式，责任链模式，访问者模式等。
- 结构型
  - 涉及如何组合类和对象以形成更大的结构，和类有关的结构型模式涉及如何合理地使用继承机制，和对象有关的结构型模式涉及如何合理地使用对象组合机制。例如：装饰模式等

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

创建型模式分为以下几种

- **单例（Singleton）模式**：某个类只能生成一个实例，该类提供了一个全局访问点供外部获取该实例，其拓展是有限多例模式。
- 原型（Prototype）模式：将一个对象作为原型，通过对其进行复制而克隆出多个和原型类似的新实例。
- **工厂方法（FactoryMethod）模式**：定义一个用于创建产品的接口，由子类决定生产什么产品。
- **抽象工厂（AbstractFactory）模式**：提供一个创建产品族的接口，其每个子类可以生产一系列相关的产品。
- 建造者（Builder）模式：将一个复杂对象分解成多个相对简单的部分，然后根据不同需要分别创建它们，最后构建成该复杂对象。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

结构型模式分为以下 7 种

1. 代理（Proxy）模式：为某对象提供一种代理以控制对该对象的访问。即客户端通过代理间接地访问该对象，从而限制、增强或修改该对象的一些特性。
2. **适配器（Adapter）模式：**将一个类的接口转换成客户希望的另外一个接口，使得原本由于接口不兼容而不能一起工作的那些类能一起工作。
3. 桥接（Bridge）模式：将抽象与实现分离，使它们可以独立变化。它是用组合关系代替继承关系来实现的，从而降低了抽象和实现这两个可变维度的耦合度。
4. **装饰（Decorator）模式：**动态地给对象增加一些职责，即增加其额外的功能。
5. **外观（Facade）模式：**为多个复杂的子系统提供一个一致的接口，使这些子系统更加容易被访问。
6. 享元（Flyweight）模式：运用共享技术来有效地支持大量细粒度对象的复用。
7. 组合（Composite）模式：将对象组合成树状层次结构，使用户对单个对象和组合对象具有一致的访问性。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

行为型模式是 GoF 设计模式中最为庞大的一类，它包含以下 11 种模式

1. 模板方法（Template Method）模式：定义一个操作中的算法骨架，将算法的一些步骤延迟到子类中，使得子类在可以不改变该算法结构的情况下重定义该算法的某些特定步骤。
2. **策略（Strategy）模式**：定义了一系列算法，并将每个算法封装起来，使它们可以相互替换，且算法的改变不会影响使用算法的客户。
3. 命令（Command）模式：将一个请求封装为一个对象，使发出请求的责任和执行请求的责任分割开。
4. **职责链（Chain of Responsibility）模式**：把请求从链中的一个对象传到下一个对象，直到请求被响应为止。通过这种方式去除对象之间的耦合。
5. 状态（State）模式：允许一个对象在其内部状态发生改变时改变其行为能力。
6. **观察者（Observer）模式**：多个对象间存在一对多关系，当一个对象发生改变时，把这种改变通知给其他多个对象，从而影响其他对象的行为。

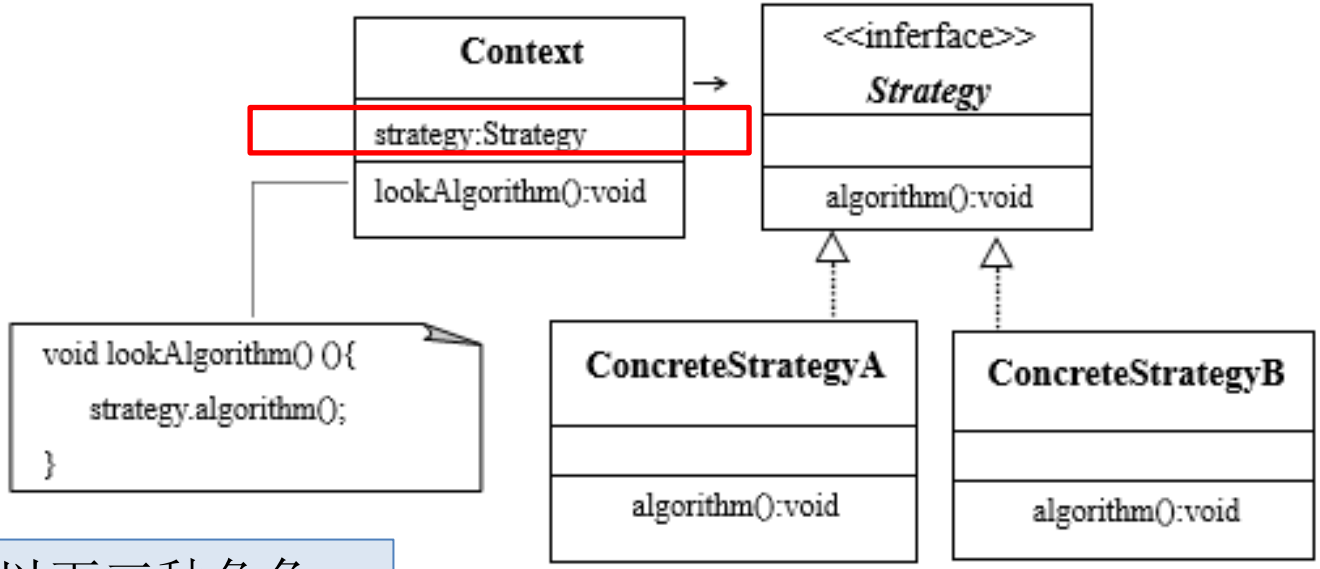
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

行为型模式是 GoF 设计模式中最为庞大的一类，它包含以下 11 种模式

7. 中介者（Mediator）模式：定义一个中介对象来简化原有对象之间的交互关系，降低系统中对象间的耦合度，使原有对象之间不必相互了解。
8. 迭代器（Iterator）模式：提供一种方法来顺序访问聚合对象中的一系列数据，而不暴露聚合对象的内部表示。
9. **访问者（Visitor）模式：**在不改变集合元素的前提下，为一个集合中的每个元素提供多种访问方式，即每个元素有多个访问者对象访问。
10. 备忘录（Memento）模式：在不破坏封装性的前提下，获取并保存一个对象的内部状态，以便以后恢复它。
11. 解释器（Interpreter）模式：提供如何定义语言的文法，以及对语言句子的解释方法，即解释器。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

策略模式



结构中包含以下三种角色：

策略（Strategy）：策略是一个接口，该接口定义若干个算法标识，即定义了若干个抽象方法。

上下文（Context）：上下文是依赖于策略接口的类（是面向策略设计的类），即上下文包含有用策略声明的变量。上下文中提供一个方法，该方法委托策略变量调用具体策略所实现的策略接口中的方法。

具体策略（ConcreteStrategy）：具体策略是实现策略接口的类。具体策略实现策略接口所定义的抽象方法，即给出算法标识的具体算法。

策略模式结构

- 策略（Strategy）：核心就是将类中经常需要变化的部分分割出来，并将每种可能的变化对应地交给抽象类的一个子类或实现接口的一个类去负责，从而让类的设计者不去关心具体实现，避免所设计的类依赖于具体的实现。
- 上下文（Context）：上下文（Context）面向策略，即是面向接口**Strategy**的类。
- 具体策略：具体策略是实现Strategy接口的类，即必须重写接口中的方法。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

案例

- 问题：在多个裁判负责打分的比赛中，每位裁判给选手一个得分，选手的最后得分是根据全体裁判的得分计算出来的。请给出几种计算选手得分的评分方案（策略），对于某次比赛，可以从你的方案中选择一种方案作为本次比赛的评分方案。
  - 在这里我们把策略接口命名为：**Strategy**。在具体应用中，这个角色的名字可以根据具体问题来命名。
  - 在本问题中将上下文命名为AverageScore，即让AverageScore类依赖于Strategy接口。
  - 每个具体策略负责一系列算法中的一个。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

1. 策略（Strategy）Strategy.java

```
public interface Strategy {
  public double computerAverage(double [] a);
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

2. 上下文（Context）AverageScore.java

```
public class  AverageScore{
    Strategy strategy;
    public void setStrategy(Strategy strategy){
        this.strategy=strategy;
    }
    public double getAverage (double [] a){
      if(strategy!=null)
        return strategy.computerAverage(a);
      else {
        System.out.println("没有求平均值的算法,得到的-1不代表平均值");
        return -1;
      }
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

3. 具体策略StrategyA.java

```
public class StrategyA implements Strategy{
    public double computerAverage(double [] a){
        double score=0,sum=0;
        for(int i=0;i<a.length;i++){
            sum=sum+a[i];
        }
        score=sum/a.length;
        return score;
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

4. 具体策略StrategyB.java

```
import java.util.Arrays;
public class StrategyB implements Strategy{
    public double computerAverage(double [] a){
        if(a.length<=2)
            return 0;
        double score=0,sum=0;
        Arrays.sort(a);   //排序数组
        for(int i=1;i<a.length-1;i++){
            sum=sum+a[i];
        }
        score=sum/(a.length-2);
        return score;
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

5. 模式的使用

```
class Person{
   String name;
   double score;
   public void setScore(double t){
      score=t;
   }
   public void setName(String s){
      name=s;
   }
   public double getScore(){
      return score;
   }
   public String getName(){
      return name;
   }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

5. 模式的使用

```
public class Application{
  public static void main(String args[]){
    AverageScore game=new AverageScore();//上下文对象game
    game.setStrategy(new StrategyA());   //上下文对象使用具体策略
    Person zhang=new Person();
    zhang.setName("张三");
    double [] a={9.12,9.25,8.87,9.99,6.99,7.88};
    double aver = game.getAverage(a) ;    //上下文对象得到平均值
    zhang.setScore(aver);
    System.out.println("算法A:");
    System.out.printf("%s最后得分:%5.3f%n",zhang.getName(),zhang.getScore());
    game.setStrategy(new StrategyB());
    aver = game.getAverage(a) ;          //上下文对象得到平均值
    zhang.setScore(aver);
    System.out.println("算法B:");
    System.out.printf("%s最后得分:%5.3f%n",zhang.getName(),zhang.getScore());
  }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

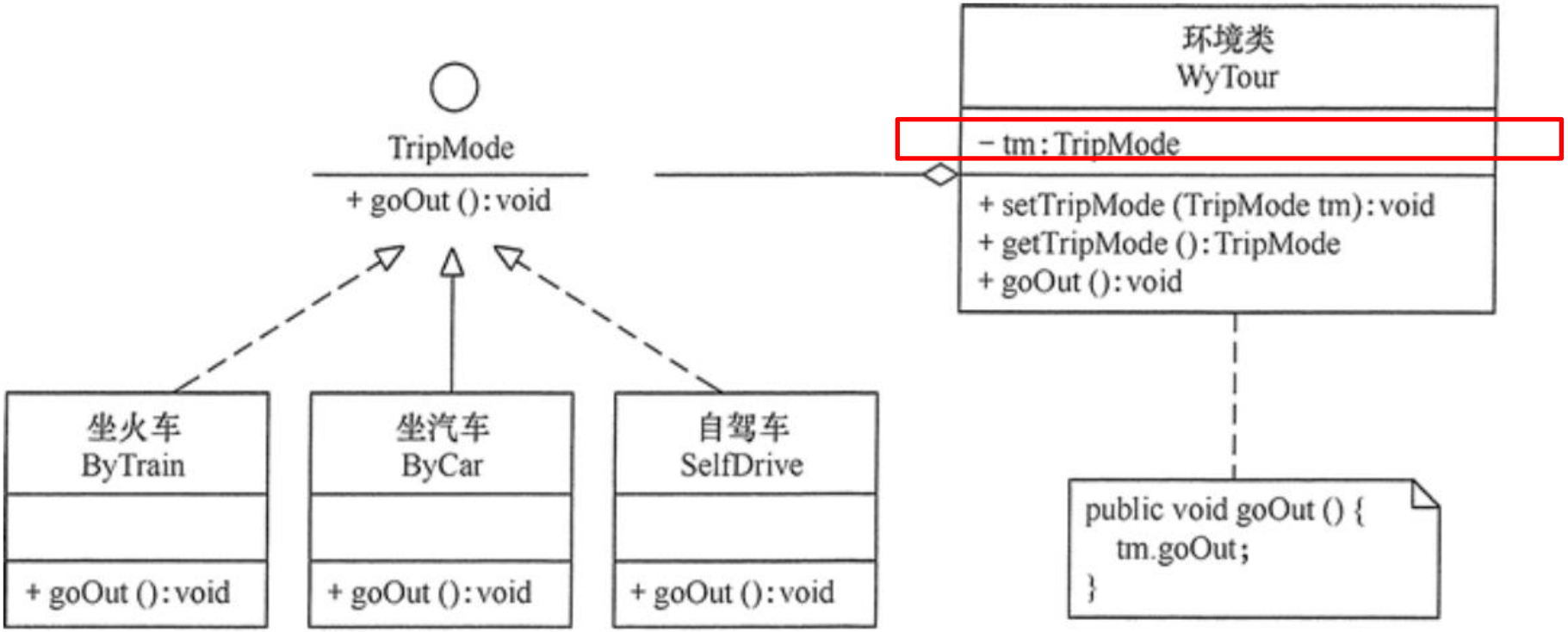
5. 模式的使用

- 使用策略模式给出了裁判给选手打分的相关类，可以将这些类看作是一个小框架，用户就可以使用这个小框架中的类编写应用程序了。相似的例子还有公司绩效考核等。

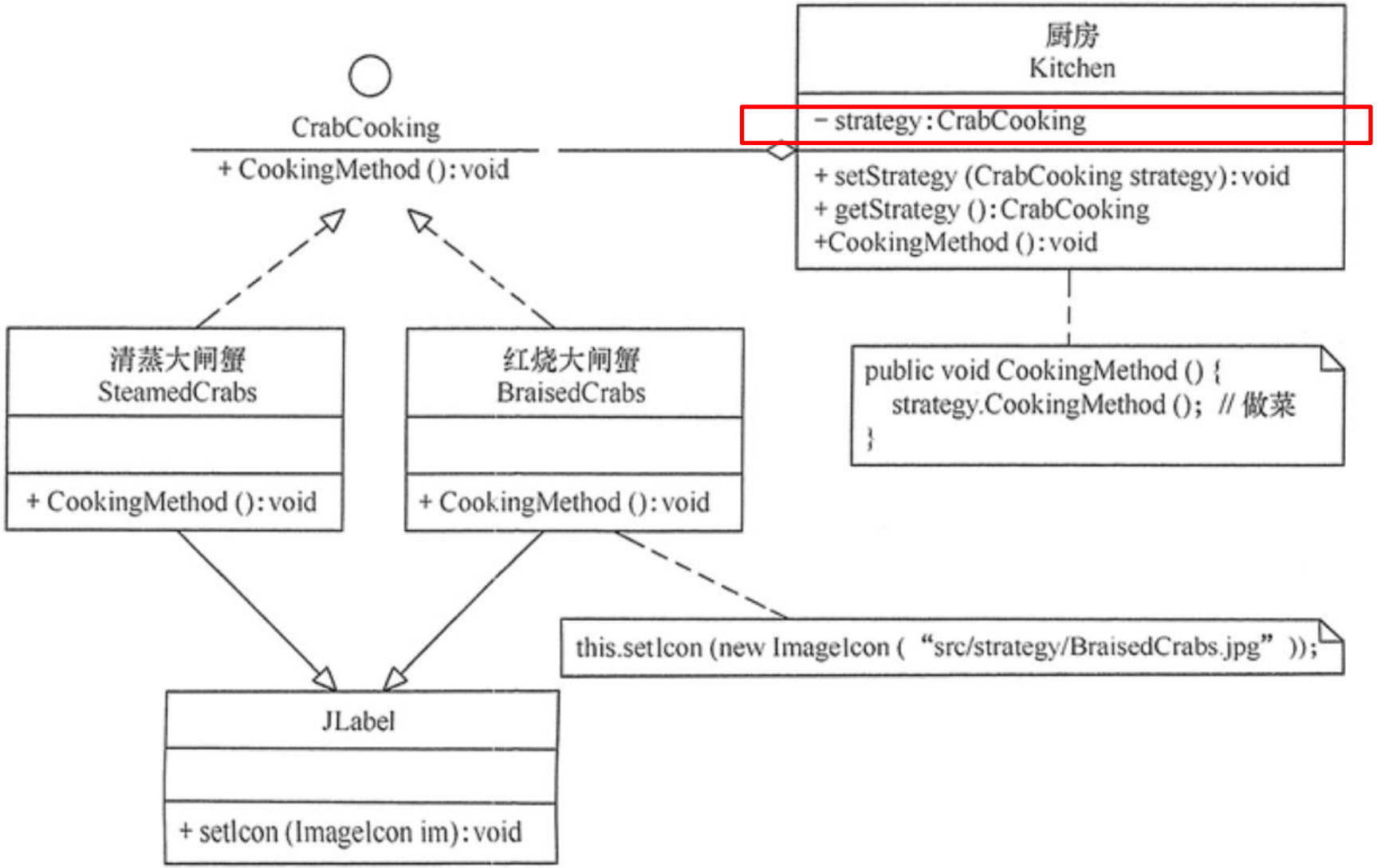
```
算法A:
张三最后得分:8.683
算法B:
张三最后得分:8.780
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

案例


TripMode
+ goOut ():void
环境类
WyTour
- tm:TripMode
+ setTripMode (TripMode tm):void
+ getTripMode ():TripMode
+ goOut ():void
坐火车
ByTrain
+ goOut ():void
坐汽车
ByCar
+ goOut ():void
自驾车
SelfDrive
+ goOut ():void
public void goOut () {
tm.goOut;
}

案例


CrabCooking
+ CookingMethod ():void
厨房
Kitchen
- strategy:CrabCooking
+ setStrategy (CrabCooking strategy):void
+ getStrategy ():CrabCooking
+CookingMethod ():void
清蒸大闸蟹
SteamedCrabs
+ CookingMethod ():void
红烧大闸蟹
BraisedCrabs
+ CookingMethod ():void
public void CookingMethod () {
strategy.CookingMethod (); // 做菜
}
this.setIcon (new ImageIcon ("src/strategy/BraisedCrabs.jpg"));
JLabel
+ setIcon (ImageIcon im):void

模式的优点

- 上下文（Context）和具体策略（ConcreteStrategy）是松耦合关系。因此上下文只知道它要使用某一个实现Strategy接口类的实例，但不需要知道具体是哪一个类。

- 策略模式满足“开-闭原则”。当增加新的具体策略时，不需要修改上下文类的代码，上下文就可以引用新的具体策略的实例。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

适合的场景

- 程序的主要类（相当于上下文角色）不希望暴露复杂的、与算法相关的数据结构，那么可以使用策略模式封装算法，即将算法分别封装到具体策略中

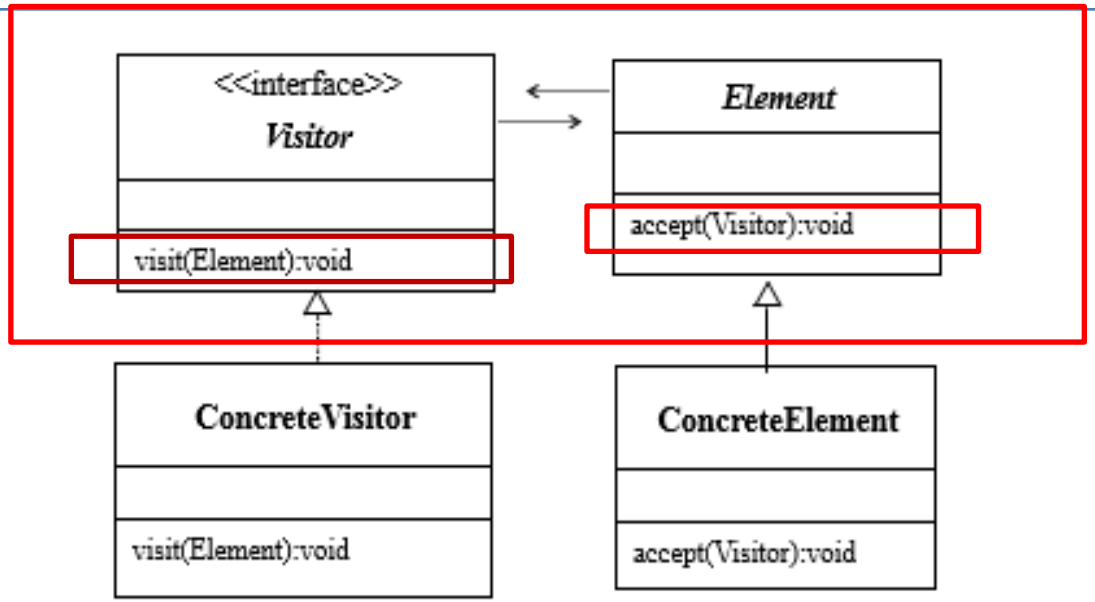
北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

相对继承机制的优势

- 考虑到系统扩展性和复用性，就应当注意面向对象的一个基本原则之一：少用继承，多用组合。
- 策略模式的应用层次采用的是组合结构，即将上下文类的某个方法的内容的不同变体分别封装在不同的具体策略中。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

访问者模式

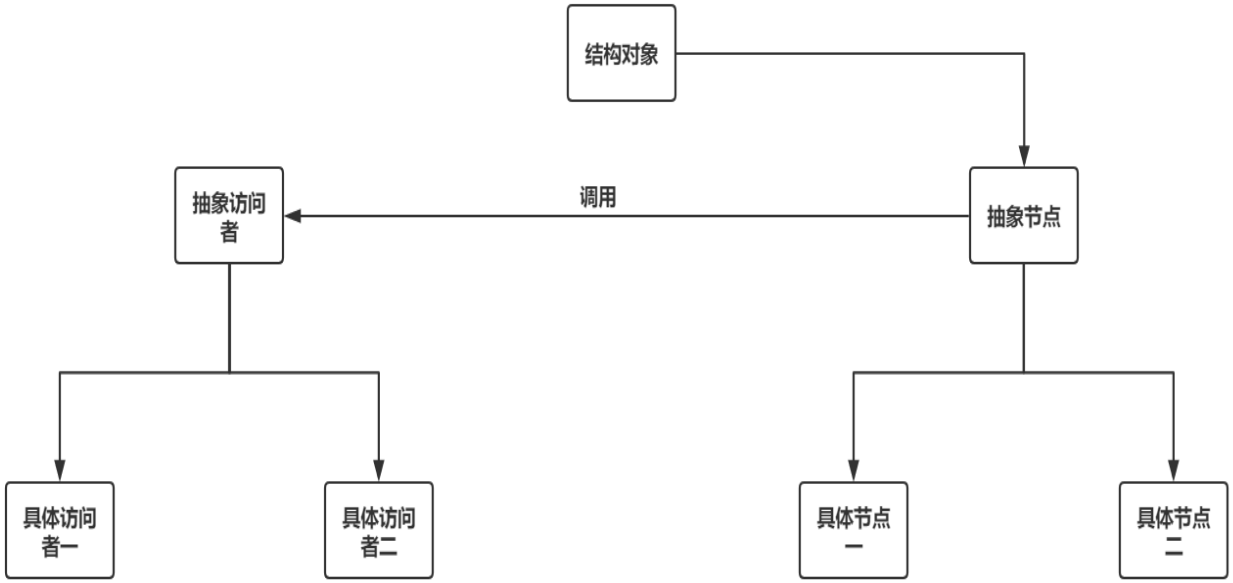


结构中包含以下4种角色：

- **抽象元素**（Element）：一个抽象类，该类定义了接收访问者的accept操作。
- **具体元素**（Concrete Element）：Element的子类。
- **抽象访问者**（Visitor）：一个接口，该接口定义操作具体元素的方法。
- **具体访问者**（Concrete Visitor）：实现Visitor接口的类。

场景1

- 门诊部是一个类似于访问者的对象，它可以访问不同类型的病人对象，例如普通病人、急诊病人、儿科病人等。
- 不同类型的病人对象可以有不同的处理方法，例如看病、输液、检查等。
- 门诊部可以对不同类型的病人对象进行不同的操作，而不需要改变病人对象的类层次结构。



北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

场景2

- 汽车修理厂的维修工可以看作是一个访问者对象，它可以访问不同类型的汽车对象，例如小轿车、越野车、卡车等。
- 不同类型的汽车对象可以有不同的故障和修理方法，例如更换零件、修补车身、调整发动机等。
- 维修工可以对不同类型的汽车对象进行不同的修理方法，而不需要改变汽车对象的类层次结构。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

抽象访问者

```
interface Visitor {
    void visit(Patient patient);
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

具体访问者

```
class Doctor implements Visitor {
    @Override
    public void visit(Patient patient) {
        System.out.println("医生为病人： " +
patient.getName() + " 处理 " + patient.getIllness());
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

抽象元素

```
interface Element {
    void accept(Visitor visitor);
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

具体元素

```
class Patient implements Element {
    private String name;
    private String illness;

    public Patient(String name, String illness) {
        this.name = name;
        this.illness = illness;
    }

    public String getName() {
        return name;
    }

    public String getIllness() {
        return illness;
    }

    @Override
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

结构对象

```
class ObjectStruture {
    private List<Element> elements = new ArrayList<>();

    public void add(Element element) {
        elements.add(element);
    }

    public void remove(Element element) {
        elements.remove(element);
    }

    public void accept(Visitor visitor) {
        for (Element element : elements) {
            element.accept(visitor);
        }
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

测试案例

```
public class Test {
    public static void main(String[] args) {
        Patient p1 = new Patient("张三", "感冒");
        Patient p2 = new Patient("李四", "发烧");

        ObjectStruture objectStruture = new ObjectStruture();
        objectStruture.add(p1);
        objectStruture.add(p2);

        Doctor doctor = new Doctor();
        objectStruture.accept(doctor);
    }
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

案例2

- 问题：根据电表显示的用电量计算用户的电费。用户包括居民和企业。访问同一个电表，即分别按家用电标准和工业用电标准计算电费。
  - 抽象的访问者：
  - 具体的访问者：
  - 抽象元素：抽象类AmmeterElement
  - 具体元素：Ammeter(模拟电表)

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

1. 抽象访问者（Visitor）

```
public interface Visitor{
  public double visit(AmmeterElement elment);
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

2. 抽象元素（Element）

```
public abstract class AmmeterElement{
  public abstract void accept(Visitor v);
  public abstract double showElectricAmount();
  public abstract void setElectricAmount(double n);
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

3. 具体访问者（Concrete Visitor）

```
public class HomeAmmeterVisitor implements Visitor{
    public double visit(AmmeterElement ammeter){
      double charge=0;
      double unitOne=0.6,unitTwo=1.05;
      int basic = 6000;
      double n=ammeter.showElectricAmount();
      if(n<=basic) {
          charge = n*unitOne;
      }
      else {
         charge =basic*unitOne+(n-basic)*unitTwo;
      }
      return charge;
    }
}
```

3. 具体访问者（Concrete Visitor）

```
public class IndustryAmmeteVisitor implements Visitor{
  public double visit(AmmeterElement ammeter){
    double charge=0;
    double unitOne=1.52,unitTwo=2.78;
    int basic = 15000;
    double n=ammeter.showElectricAmount();
    if(n<=basic) {
      charge = n*unitOne;
    }
    else {
      charge =basic*unitOne+(n-basic)*unitTwo;
    }
    return charge;
  }
}
```

4. 具体元素（Concrete Element）

```
public class Ammeter extends AmmeterElement{
  double electricAmount;    //电表的电量
  public void setElectricAmount(double n) {
    electricAmount = n;
  }
  public void accept(Visitor visitor){
    double feiyong=visitor.visit(this); //让访问者访问当前元素
    System.out.println("当前电表的用户需要交纳电费:"+feiyong+"元");
  }
  public double showElectricAmount(){
    return electricAmount;
  }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

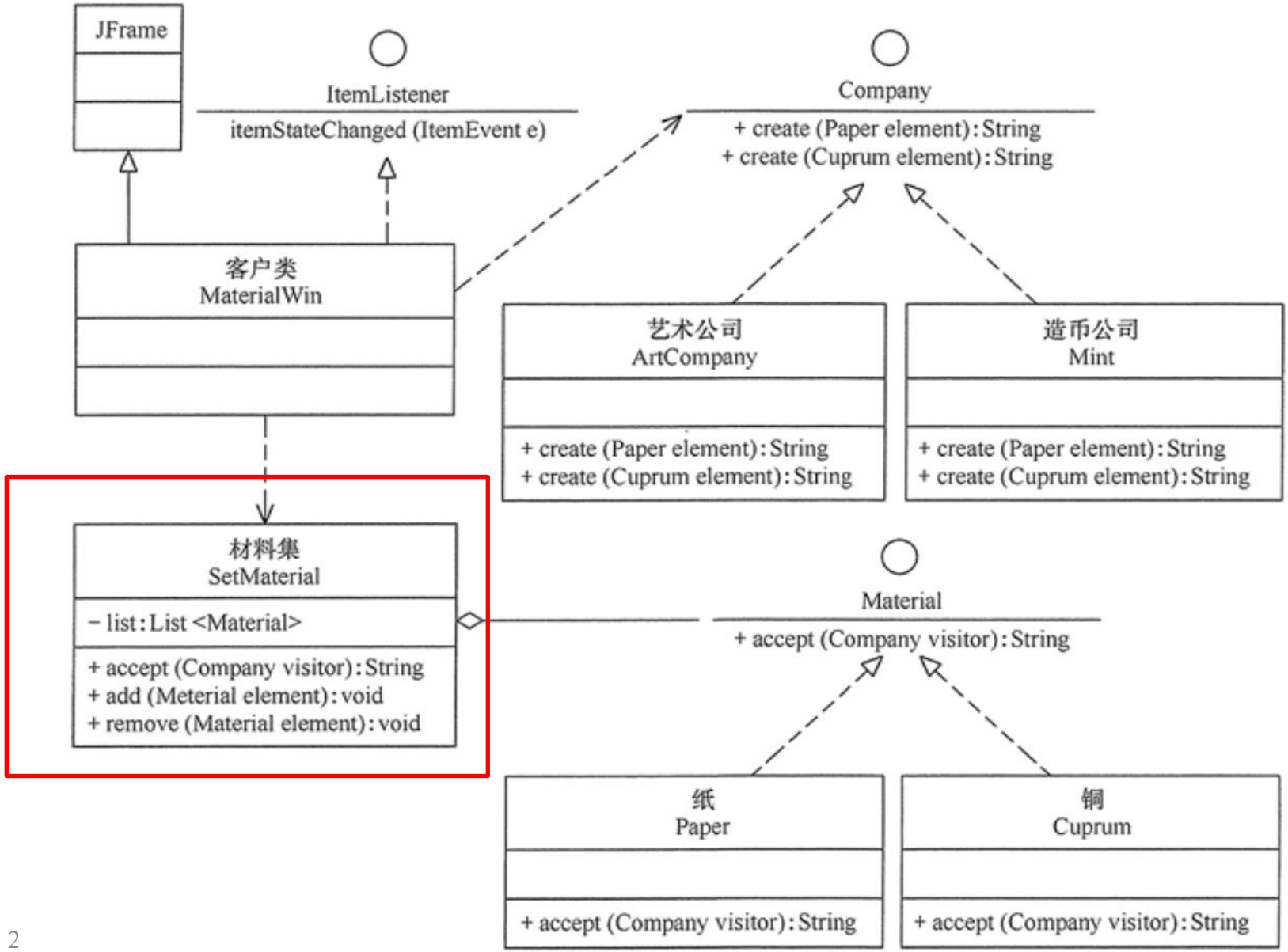
模式的使用：

```
public class Application{
  public static void main(String args[]) {
    Visitor 计表员=new HomeAmmeterVisitor(); //按家用电标准计算电费的"计表员"
    Ammeter 电表=new Ammeter();
    电表.setElectricAmount(5678);
    电表.accept(计表员);
    计表员=new IndustryAmmeteVisitor(); //按工业用电标准计算电费的"计表员"
    电表.setElectricAmount(5678);
    电表.accept(计表员);
  }
}
```

```
当前电表的用户需要交纳电费:3406.7999999999997元
当前电表的用户需要交纳电费:8630.56元
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

案例：艺术公司和造币公司结构图



航天大學
COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

模式的使用：

利用访问者模式设计艺术公司和造币公司
原材料有：铜和纸，请选择生产公司：
艺术公司
造币公司

利用访问者模式设计艺术公司和造币公司
中国人民银行
100
壹佰圆
原材料有：铜和纸，请选择生产公司：
艺术公司
造币公司

模式优点和使用场景

- 模式优点：在不改变一个集合中的元素的类的情况下，可以增加新的施加于该元素上的新操作。保持一定的扩展性。
- 使用场景：需要对集合中的对象进行很多不同的并且不相关的操作，而我们又不想修改对象的类，就可以使用访问者模式。访问者模式可以在Visitor类中集中定义一些关于集合中对象的操作。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院